

RISC-V Processor with Custom AES Instruction Set

3EC503CC24-Computer Architecture

Special Assignment Report



INSTITUTE OF TECHNOLOGY , NIRMA UNIVERSITY

FACULTY : Dr. Dhaval Shah

Submitted By : Jiyan Patel (23BEC137) and Mihir Patel (23BEC141)

Abstract :

With the rapid growth of Internet of Things (IoT) devices and embedded systems, secure communication has become an essential requirement. Cryptographic algorithms such as Advanced Encryption Standard (AES) are widely used to ensure confidentiality and data integrity. However, implementing AES purely in software on general-purpose processors results in higher latency and increased computational overhead. This project presents the design and implementation of a single-cycle RISC-V processor integrated with a hardware AES cryptographic accelerator. The processor is implemented in Verilog HDL and supports both standard RISC-V instructions and custom AES instructions. Dedicated hardware modules are designed for AES operations such as SubBytes, ShiftRows, MixColumns, and AddRoundKey. The integration of AES functionality inside the processor significantly improves encryption performance and reduces execution cycles compared to software implementations. Simulation results demonstrate the correctness of the processor operation and successful encryption using AES. The proposed architecture provides a secure and efficient solution for embedded and IoT applications requiring hardware-accelerated cryptography.

Keywords :

RISC-V, AES Encryption, Cryptographic Processor, Hardware Acceleration, Verilog HDL, Embedded Security

1. Introduction:

Modern embedded systems require both high performance and strong security mechanisms. As data transmission increases across networks and IoT devices, protecting sensitive information becomes critical. Encryption algorithms are widely used to secure data against unauthorized access.

The Advanced Encryption Standard (AES) is one of the most widely adopted symmetric encryption algorithms used in applications such as wireless communication, banking systems, and secure storage. While AES can be implemented in software, it consumes considerable processing time when executed on general-purpose processors.

The RISC-V architecture is an open-source instruction set architecture that has gained significant popularity due to its flexibility, simplicity, and extensibility. One of its major advantages is the ability to introduce custom instructions, which allows designers to integrate specialized hardware accelerators.

This project focuses on designing a single-cycle RISC-V processor enhanced with AES cryptographic instructions. The AES operations are implemented as dedicated hardware modules to accelerate encryption operations.

2. Literature Survey / State of the Art :

Several researchers have proposed hardware-accelerated cryptographic processors to improve security performance.

1. RISC-V Cryptographic Extensions

Researchers have developed cryptographic instruction extensions for RISC-V that enable efficient implementation of AES and other algorithms. These extensions reduce the number of instructions required for encryption operations.

2. Hardware Implementation of AES

Many FPGA-based implementations of AES have been proposed to achieve high throughput and low latency. Hardware implementations typically perform AES operations in parallel, resulting in faster execution compared to software.

3. Embedded Cryptographic Processors

Modern embedded processors often integrate cryptographic accelerators to enhance security. These processors offload encryption tasks from the CPU to dedicated hardware units.

4. FPGA-Based Secure Systems

Field Programmable Gate Arrays (FPGAs) are widely used to implement secure processors because they provide flexibility, reconfigurability, and parallel processing capabilities.

The existing research demonstrates that integrating cryptographic accelerators within processors significantly improves performance and efficiency.

3. Limitations of Existing Technologies:

Despite the advancements in cryptographic processors, several limitations still exist:

1. High computational overhead when AES is implemented purely in software.
2. Increased execution time due to multiple instruction cycles required for encryption operations.
3. Higher power consumption in software-based cryptographic implementations.

4. Many existing architectures require complex pipelined designs, which increase design complexity.
- 5.

4. Methodology:

Overall System Architecture

The processor follows a single-cycle architecture, where every instruction completes in one clock cycle. The main functional blocks implemented in the design include:

1. Program Counter (PC)
2. Instruction Memory
3. Control Unit
4. Register File
5. Immediate Generator
6. ALU
7. AES Cryptographic Accelerator
8. Data Memory
9. Multiplexers and Control Signals

Each of these modules is coded as an independent Verilog module and connected through a datapath.

Program Counter (PC)

The Program Counter stores the address of the current instruction being executed. Initially set to zero. After every clock cycle, it increments by 4 bytes to fetch the next instruction. PC provides the address to the instruction memory.

Instruction Memory

Instruction memory stores the RISC-V program instructions.

Operation:

The PC provides the address.

The instruction stored at that address is fetched.

Instruction Format

RISC-V instructions are 32-bit wide

31-25 funct7

24-20 rs2

19-15 rs1

14-12 funct3

11-7 rd

6-0 opcode

Control Unit

The Control Unit decodes the instruction opcode and generates control signals for other blocks.

Inputs to control unit are opcode, funct7 and funct3.

Outputs are control signals such as : RegWrite , MemRead, MemWrite, ALUSrc, MemtoReg, Branch, AES_Enable.

Register File

The register file stores temporary data used during instruction execution.

Specifications

32 registers

Each register is 32 bits

Two read ports

One write port

Inputs

rs1 (source register 1)

rs2 (source register 2)

rd (destination register)

Outputs

Data1

Data2

This values are sent to ALU or AES module

Immediate Generator

Some instructions use immediate values instead of registers. The immediate generator extracts the immediate bits from the instruction and converts them into 32-bit values.

Arithmetic Logic Unit (ALU)

The ALU performs arithmetic and logical operations.

The ALU operation depends on:

opcode

funct3

Funct7

AES Cryptographic Accelerator

The AES module performs hardware-accelerated encryption operations.

Instead of executing AES steps through many instructions, the processor executes a single custom instruction.

AES Functional Modules

The AES accelerator contains the following blocks:

SubBytes module
ShiftRows module
MixColumns module
AddRoundKey module

These modules are connected sequentially.

SubBytes Module

Performs byte substitution using the AES S-box lookup table.

Input: 8-bit data

Output: substituted byte.

This module is implemented using a case statement lookup table in Verilog.

ShiftRows Module

This module performs row shifting on the 4×4 AES state matrix.

Row 0 - shift 0

Row 1 - shift 1 byte

Row 2 - shift 2 byte

Row 3 - shift 3 byte

MixColumns Module

AddRoundKey Module

AddRoundKey performs XOR operation between the state and the round key.

State = State XOR RoundKey

This step integrates the secret key into the encryption process.

Key Expansion (Key Schedule)

AES encryption requires a different round key for each round.

The Key Expansion module generates these round keys from the original secret key.

For AES-128:

Input key = 128 bits

Total rounds = 10

Total round keys = 11

Key expansion uses three main operations:

1. RotWord

$[a_0 \ a_1 \ a_2 \ a_3] \rightarrow [a_1 \ a_2 \ a_3 \ a_0]$

2. SubWord

Apply S-box substitution to each byte.

3. Rcon

Round constant added using XOR.

Key Expansion Formula

For AES-128:

$W[i] = W[i-4] \text{ XOR } g(W[i-1]) \quad (\text{when } i \bmod 4 = 0)$

Where:

$$g() = \text{RotWord} + \text{SubWord} + \text{Rcon}$$

Data Memory:

Data memory stores and retrieves data during program execution.

Supported operations are lw and sw. And control signals needed are MemRead and MemWrite.

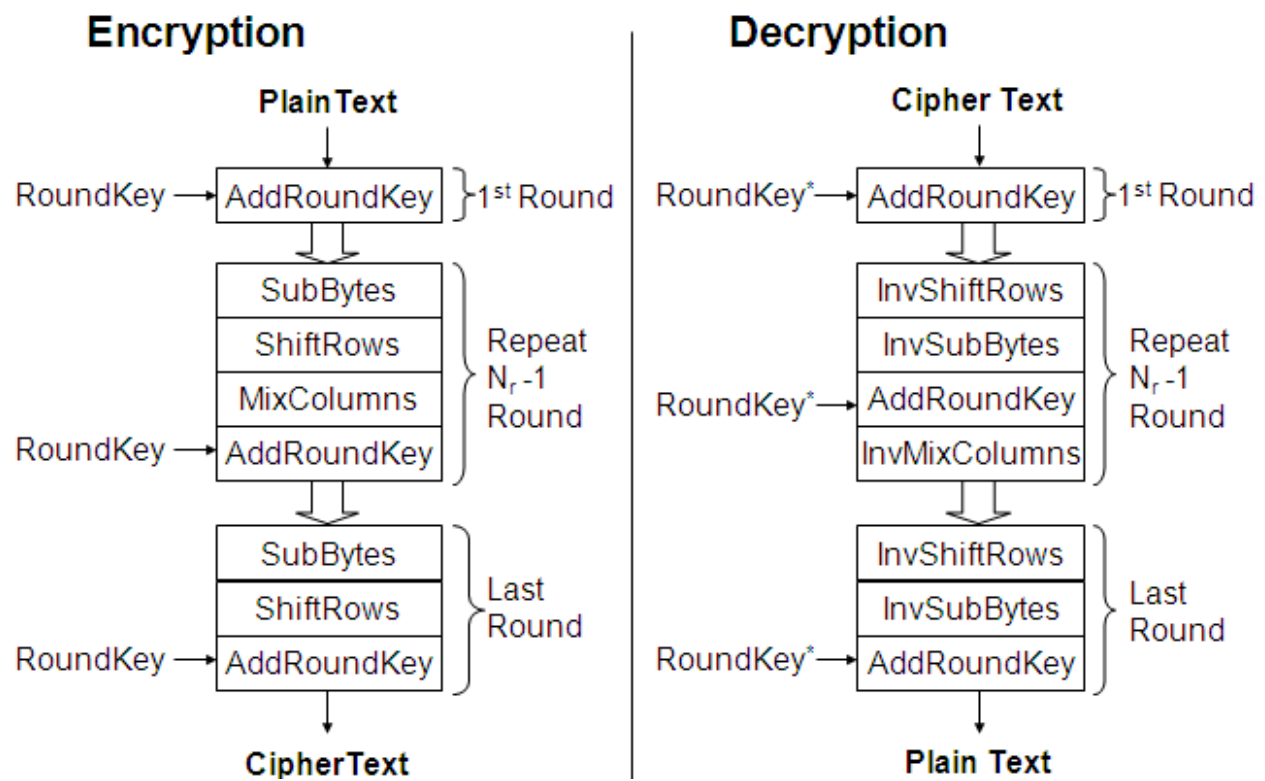
Processor Execution Flow

The processor executes instructions using the following sequence:

1. Fetch instruction from instruction memory
2. Decode instruction using control unit
3. Read registers from register file
4. Execute operation in ALU or AES module
5. Access memory if required
6. Write result back to register file

This process completes in one clock cycle for each instruction.

5. AES Encryption and Decryption block diagram



6. Simulation Results:

The screenshot displays the EDA Playground interface with two tabs: 'testbench.v' and 'program.hex'. The 'testbench.v' tab shows a Verilog testbench for a control unit, including display statements for register and memory values. The 'program.hex' tab shows the corresponding hex file. The simulation log at the bottom indicates the simulation is finished at 510 ns.

```
testbench.v
490 //
491 end
492
493 always @(posedge clk) begin
494     $display("\n--- Register File ---");
495
496     $display("x1 = %d", uut.RF.registers[1]);
497     $display("x2 = %d", uut.RF.registers[2]);
498     $display("x3 = %d", uut.RF.registers[3]);
499     $display("x4 = %d", uut.RF.registers[4]);
500     $display("x5 = %d", uut.RF.registers[5]);
501     $display("x6 = %d", uut.RF.registers[6]);
502     $display("x7 = %d", uut.RF.registers[7]);
503 end
504
505 always @(posedge clk) begin
506     $display("\n--- Data Memory ---");
507
508     $display("MEM[0] = %d", uut.DM.memory[0]);
509     $display("MEM[1] = %d", uut.DM.memory[1]);
510     $display("MEM[2] = %d", uut.DM.memory[2]);
511     $display("MEM[3] = %d", uut.DM.memory[3]);
512 end
513 end
```

```
program.hex
888 module control_unit(
889     input [6:0] opcode,
890     output reg RegWrite,
891     output reg MemRead,
892     output reg MemWrite,
893     output reg ALUSrc,
894     output reg Branch,
895     output reg Jump,
896     output reg LUI,
897     output reg AES_EN
898 );
899
900 always @(*) begin
901     case(opcode)
902         7'b0110011: begin RegWrite=1; MemRead=0; MemWrite=0; ALUSrc=0;
903             Branch=0; Jump=0; LUI=0; AES_EN=0; end
904         7'b0010011: begin RegWrite=1; MemRead=0; MemWrite=0; ALUSrc=1;
905             Branch=0; Jump=0; LUI=0; AES_EN=0; end
906         7'b0000011: begin RegWrite=1; MemRead=1; MemWrite=0; ALUSrc=1;
907             Branch=0; Jump=0; LUI=0; AES_EN=0; end
908         7'b1000011: begin RegWrite=0; MemRead=0; MemWrite=1; ALUSrc=1;
909             Branch=0; Jump=0; LUI=0; AES_EN=0; end
910         7'b1100011: begin RegWrite=1; MemRead=0; MemWrite=0; ALUSrc=0;
911             Branch=1; Jump=0; LUI=0; AES_EN=0; end
912         7'b1101111: begin RegWrite=1; MemRead=0; MemWrite=0; ALUSrc=0;
913             Branch=0; Jump=1; LUI=0; AES_EN=0; end
914         7'b0110111: begin RegWrite=1; MemRead=0; MemWrite=0; ALUSrc=1;
915             Branch=0; Jump=0; LUI=1; AES_EN=0; end
916         7'b0001011: begin RegWrite=1; MemRead=0; MemWrite=0; ALUSrc=0;
917             Branch=0; Jump=0; LUI=0; AES_EN=1; end
918     endcase
919 end
```

Log:

```
495000 000000cc xxxxxxxx
--- Register File ---
x1 = 5
x2 = 10
x3 = 15
x4 = 5
x5 = 15
x6 = x
x7 = 2
--- Data Memory ---
MEM[0] = 15
MEM[1] = x
MEM[2] = x
MEM[3] = x
$S05000 000000d0 xxxxxxxx
Simulation Finished
Simulation complete via $finish(1) at time 510 NS + 0
./testbench.v:39 $finish;
xcelium> exit
```

This is the result after writing testbench to check processor output by adding hex file of instructions the instructions tested here are ADDI, ADD, SUB, SW, LW, BEQ, JAL

#Inputs of hex file

00500093

00a00113

002081b3

40110233

00302023

00002283

00328463

00100313

0080006f

00200393

ADDI x1, x0, 5

ADDI x2, x0, 10

ADD x3, x1, x2

SUB x4, x2, x1

LW x5, 0(x0)

BEQ x5, x3, label

ADDI x6, x0, 1

JAL x0, end

ADDI x7, x0, 2

TESTING AES BLOCK:

The screenshot displays the EDA Playground interface. On the left, a sidebar lists various tools and simulators, with Cadence Xcelium 25.03 selected. The main workspace is divided into two panels. The left panel, titled 'testbench.sv', contains Verilog code for a testbench that instantiates an AES block and performs a single-cycle encryption. The right panel, titled 'design.sv', contains the Verilog code for the AES block itself, which implements the AES-128 encryption algorithm using a series of rounds. Below the code panels, a 'Log' window shows the simulation results, including the expected AES-128 ciphertext and the actual output of the encryption process. The simulation is complete, and the output matches the expected ciphertext.

```
41
42 // AES MIXCOLUMNS
43 aes_op = 3'b011;
44 #10;
45 $display("AES MIXCOLUMNS OUTPUT = %h", out);
46
47
48 // AES ADDROUNDKEY
49 aes_op = 3'b100;
50 #10;
51 $display("AES ADDROUNDKEY OUTPUT = %h", out);
52
53
54 // FULL AES ENCRYPTION
55 aes_op = 3'b101;
56 #20;
57 $display("AES ENCRYPT OUTPUT = %h", out);
58
59
60 $display("-----");
61 $display("Expected AES-128 Ciphertext:");
62 $display("69c4e0d86a7b0430d8cd78070b4c55a");
63 $display("-----");
64
65 $finish;
66
67
68 endmodule
```

```
20 aes_encrypt u1(data, round_keys, enc_out);
21
22 always @(*) begin
23   case(aes_op)
24     3'b001: out = sub_out; // aessub
25     3'b010: out = shift_out; // aesshift
26     3'b011: out = mix_out; // aesmix
27     3'b100: out = data ^ key; // aeskey
28     3'b101: out = enc_out; // aesenc
29   default: out = 128'b0;
30 endcase
31 end
32
33 module aes_encrypt
34   input [127:0] data,
35   input [1407:0] round_keys,
36   output [127:0] out
37 );
38
39 wire [127:0] r0,r1,r2,r3,r4,r5,r6,r7,r8,r9,r10;
40 assign r0 = data ^ round_keys[1407:1280];
```

key = 000102030405060708090a0b0c0d0e0f

AES SUBBYTES OUTPUT = 638293c31bfc33f5c4eaccd4bc12816
AES SHIFTRWS OUTPUT = 0055aaff4499ee3388dd2277cc1166bb
AES MIXCOLUMNS OUTPUT = 2277005566334411aaff88ddeebbcc99
AES ADDROUNDKEY OUTPUT = 00102030405060708090a0b0c0d0e0f0
AES ENCRYPT OUTPUT = 69c4e0d86a7b0430d8cd78070b4c55a

Expected AES-128 Ciphertext:
69c4e0d86a7b0430d8cd78070b4c55a

Simulation complete via \$finish(1) at time 60 NS + 0
./testbench.sv:65 \$finish;

Our cipher text is matching AES standardised cipher text.

CONCLUSION:

Conclusion

This project presented the design and implementation of a single-cycle RISC-V processor integrated with an AES-128 cryptographic accelerator using Verilog HDL. The processor architecture was developed by implementing essential components such as the program counter, instruction memory, register file, control unit, ALU, immediate generator, data memory, and branch logic. A subset of the RV32I instruction set including arithmetic, logical, memory access, and control instructions was successfully implemented and verified through simulation.

To enhance the processor's capability for secure applications, a dedicated AES hardware accelerator was integrated into the processor datapath. The AES module supports cryptographic operations including SubBytes, ShiftRows, MixColumns, AddRoundKey, and full AES encryption, along with a key expansion unit that generates round keys required for the AES-128 encryption process. The accelerator is activated through custom AES instructions, enabling efficient execution of encryption tasks directly in hardware.

Simulation results confirm that the processor correctly executes standard RISC-V instructions as well as AES cryptographic operations. The use of hardware acceleration significantly improves the efficiency of cryptographic computation compared to software-based implementations. The modular design of the processor also makes it scalable and suitable for future enhancements.

References

1. **National Institute of Standards and Technology (NIST)**, *Advanced Encryption Standard (AES)*, FIPS Publication 197, November 2001.
2. Joan Daemen and Vincent Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*, Springer, 2002.
3. David A. Patterson and Andrew Waterman, *The RISC-V Reader: An Open Architecture Atlas*, Strawberry Canyon LLC, 2017.
4. RISC-V International, *The RISC-V Instruction Set Manual Volume I: Unprivileged ISA Specification*, Version 20191213.
5. William Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson Education, 7th Edition, 2017.
6. IEEE Xplore Digital Library, Research papers on *FPGA Implementation of AES Encryption and RISC-V Processors*, available at <https://ieeexplore.ieee.org>.
7. OpenCores, *AES Hardware Implementation Projects*, available at <https://opencores.org>.
8. Xilinx, *Vivado Design Suite User Guide*, for FPGA-based hardware design and simulation.

CODE OF AES BLOCK :

```
module AES(  
input [127:0] data,  
input [127:0] key,  
input [2:0] aes_op,  
output reg [127:0] out  
);  
  
wire [127:0] sub_out;  
wire [127:0] shift_out;  
wire [127:0] mix_out;  
wire [127:0] enc_out;  
wire [1407:0] round_keys;  
aes_subbytes u1(data,sub_out);  
aes_shiftrows u2(data,shift_out);  
aes_mixcolumns u3(data,mix_out);  
  
aes_key_expand keyexp(key,round_keys);  
  
aes_encrypt u4(data,round_keys,enc_out);  
  
always @(*) begin  
  
case(aes_op)  
  
3'b001: out = sub_out;    // aessub  
3'b010: out = shift_out;  // aesshift  
3'b011: out = mix_out;    // aesmix  
3'b100: out = data ^ key; // aeskey  
3'b101: out = enc_out;    // aesenc  
  
default: out = 128'b0;  
  
endcase
```

```
end
```

```
endmodule
```

```
module aes_encrypt(
```

```
input [127:0] data,
```

```
input [1407:0] round_keys,
```

```
output [127:0] out
```

```
);
```

```
wire [127:0] r0,r1,r2,r3,r4,r5,r6,r7,r8,r9,r10;
```

```
assign r0 = data ^ round_keys[1407:1280];
```

```
aes_round round1(r0,round_keys[1279:1152],r1);
```

```
aes_round round2(r1,round_keys[1151:1024],r2);
```

```
aes_round round3(r2,round_keys[1023:896],r3);
```

```
aes_round round4(r3,round_keys[895:768],r4);
```

```
aes_round round5(r4,round_keys[767:640],r5);
```

```
aes_round round6(r5,round_keys[639:512],r6);
```

```
aes_round round7(r6,round_keys[511:384],r7);
```

```
aes_round round8(r7,round_keys[383:256],r8);
```

```
aes_round round9(r8,round_keys[255:128],r9);
```

```
aes_final round10(r9,round_keys[127:0],r10);
```

```
assign out = r10;
```

```
endmodule
```

```
module aes_round(
```

```
input [127:0] data,
```

```
input [127:0] key,
```

```
output [127:0] out
```

```
);
```

```
wire [127:0] sub;
```

```
wire [127:0] shift;
```

```
wire [127:0] mix;
```

```
aes_subbytes sb(data,sub);
```

```
aes_shiftrows sr(sub,shift);
```

```
aes_mixcolumns mc(shift,mix);
```

```
assign out = mix ^ key;
```

```
endmodule
```

```
module aes_final(
```

```
input [127:0] data,
```

```
input [127:0] key,
```

```
output [127:0] out
```

```
);
```

```
wire [127:0] sub;
```

```
wire [127:0] shift;
```

```
aes_subbytes sb(data,sub);
```

```
aes_shiftrows sr(sub,shift);
```

```
assign out = shift ^ key;
```

```
endmodule
```

```
module aes_subbytes(
```

```
input [127:0] in,
```

```
output [127:0] out  
);
```

```
genvar i;
```

```
generate  
for(i=0;i<16;i=i+1)  
begin:sbox
```

```
    aes_sbox s(  
        in[8*i+:8],  
        out[8*i+:8]  
    );
```

```
end
```

```
endgenerate
```

```
endmodule
```

```
module aes_subword(  
    input  [31:0] in,  
    output [31:0] out  
);
```

```
    aes_sbox s0(.a(in[31:24]), .d(out[31:24]));  
    aes_sbox s1(.a(in[23:16]), .d(out[23:16]));  
    aes_sbox s2(.a(in[15:8]),  .d(out[15:8]));  
    aes_sbox s3(.a(in[7:0]),   .d(out[7:0]));
```

```
endmodule
```

```
module aes_shiftrows(  
    input  [127:0] in,  
    output [127:0] out
```

```
);
```

```
assign out = {
```

```
in[127:120], in[87:80], in[47:40], in[7:0],  
in[95:88], in[55:48], in[15:8], in[103:96],  
in[63:56], in[23:16], in[111:104], in[71:64],  
in[31:24], in[119:112], in[79:72], in[39:32]
```

```
};
```

```
endmodule
```

```
module aes_mixcolumns(
```

```
input [127:0] in,
```

```
output [127:0] out
```

```
);
```

```
function [7:0] xtime;
```

```
input [7:0] b;
```

```
begin
```

```
xtime = (b << 1) ^ (b[7] ? 8'h1b : 8'h00);
```

```
end
```

```
endfunction
```

```
function [7:0] mul3;
```

```
input [7:0] b;
```

```
begin
```

```
mul3 = xtime(b) ^ b;
```

```
end
```

```
endfunction
```

```
function [31:0] mixcol;
```

```
input [31:0] c;
```

```
reg [7:0] a0,a1,a2,a3;
```

```
begin
```

```
// Correct byte order
```

```
a0 = c[31:24];
```

```
a1 = c[23:16];
```

```
a2 = c[15:8];
```

```
a3 = c[7:0];
```

```
mixcol[31:24] = xtime(a0) ^ mul3(a1) ^ a2 ^ a3;
```

```
mixcol[23:16] = a0 ^ xtime(a1) ^ mul3(a2) ^ a3;
```

```
mixcol[15:8] = a0 ^ a1 ^ xtime(a2) ^ mul3(a3);
```

```
mixcol[7:0] = mul3(a0) ^ a1 ^ a2 ^ xtime(a3);
```

```
end
```

```
endfunction
```

```
assign out = {
```

```
mixcol(in[127:96]),
```

```
mixcol(in[95:64]),
```

```
mixcol(in[63:32]),
```

```
mixcol(in[31:0])
```

```
};
```

```
endmodule
```

```
module aes_sbox(
```

```
input [7:0] a,
```

```
output reg [7:0] d
```

```
);
```

```
always @(*) begin
```

```
case(a)
```

8'h00:d=8'h63;8'h01:d=8'h7c;8'h02:d=8'h77;8'h03:d=8'h7b;
8'h04:d=8'hf2;8'h05:d=8'h6b;8'h06:d=8'h6f;8'h07:d=8'hc5;
8'h08:d=8'h30;8'h09:d=8'h01;8'h0a:d=8'h67;8'h0b:d=8'h2b;
8'h0c:d=8'hfe;8'h0d:d=8'hd7;8'h0e:d=8'hab;8'h0f:d=8'h76;

8'h10:d=8'hca;8'h11:d=8'h82;8'h12:d=8'hc9;8'h13:d=8'h7d;
8'h14:d=8'hfa;8'h15:d=8'h59;8'h16:d=8'h47;8'h17:d=8'hf0;
8'h18:d=8'had;8'h19:d=8'hd4;8'h1a:d=8'ha2;8'h1b:d=8'haf;
8'h1c:d=8'h9c;8'h1d:d=8'ha4;8'h1e:d=8'h72;8'h1f:d=8'hc0;

8'h20:d=8'hb7;8'h21:d=8'hfd;8'h22:d=8'h93;8'h23:d=8'h26;
8'h24:d=8'h36;8'h25:d=8'h3f;8'h26:d=8'hf7;8'h27:d=8'hcc;
8'h28:d=8'h34;8'h29:d=8'ha5;8'h2a:d=8'he5;8'h2b:d=8'hf1;
8'h2c:d=8'h71;8'h2d:d=8'hd8;8'h2e:d=8'h31;8'h2f:d=8'h15;

8'h30:d=8'h04;8'h31:d=8'hc7;8'h32:d=8'h23;8'h33:d=8'hc3;
8'h34:d=8'h18;8'h35:d=8'h96;8'h36:d=8'h05;8'h37:d=8'h9a;
8'h38:d=8'h07;8'h39:d=8'h12;8'h3a:d=8'h80;8'h3b:d=8'he2;
8'h3c:d=8'heb;8'h3d:d=8'h27;8'h3e:d=8'hb2;8'h3f:d=8'h75;

8'h40:d=8'h09;8'h41:d=8'h83;8'h42:d=8'h2c;8'h43:d=8'h1a;
8'h44:d=8'h1b;8'h45:d=8'h6e;8'h46:d=8'h5a;8'h47:d=8'ha0;
8'h48:d=8'h52;8'h49:d=8'h3b;8'h4a:d=8'hd6;8'h4b:d=8'hb3;
8'h4c:d=8'h29;8'h4d:d=8'he3;8'h4e:d=8'h2f;8'h4f:d=8'h84;

8'h50:d=8'h53;8'h51:d=8'hd1;8'h52:d=8'h00;8'h53:d=8'hed;
8'h54:d=8'h20;8'h55:d=8'hfc;8'h56:d=8'hb1;8'h57:d=8'h5b;
8'h58:d=8'h6a;8'h59:d=8'hcb;8'h5a:d=8'hbe;8'h5b:d=8'h39;
8'h5c:d=8'h4a;8'h5d:d=8'h4c;8'h5e:d=8'h58;8'h5f:d=8'hcf;

8'h60:d=8'hd0;8'h61:d=8'hcf;8'h62:d=8'haa;8'h63:d=8'hfb;
8'h64:d=8'h43;8'h65:d=8'h4d;8'h66:d=8'h33;8'h67:d=8'h85;
8'h68:d=8'h45;8'h69:d=8'hf9;8'h6a:d=8'h02;8'h6b:d=8'h7f;
8'h6c:d=8'h50;8'h6d:d=8'h3c;8'h6e:d=8'h9f;8'h6f:d=8'ha8;

8'h70:d=8'h51;8'h71:d=8'ha3;8'h72:d=8'h40;8'h73:d=8'h8f;

8'h74:d=8'h92;8'h75:d=8'h9d;8'h76:d=8'h38;8'h77:d=8'hf5;
8'h78:d=8'hbc;8'h79:d=8'hb6;8'h7a:d=8'hda;8'h7b:d=8'h21;
8'h7c:d=8'h10;8'h7d:d=8'hff;8'h7e:d=8'hf3;8'h7f:d=8'hd2;

8'h80:d=8'hcd;8'h81:d=8'h0c;8'h82:d=8'h13;8'h83:d=8'hec;
8'h84:d=8'h5f;8'h85:d=8'h97;8'h86:d=8'h44;8'h87:d=8'h17;
8'h88:d=8'hc4;8'h89:d=8'ha7;8'h8a:d=8'h7e;8'h8b:d=8'h3d;
8'h8c:d=8'h64;8'h8d:d=8'h5d;8'h8e:d=8'h19;8'h8f:d=8'h73;

8'h90:d=8'h60;8'h91:d=8'h81;8'h92:d=8'h4f;8'h93:d=8'hdc;
8'h94:d=8'h22;8'h95:d=8'h2a;8'h96:d=8'h90;8'h97:d=8'h88;
8'h98:d=8'h46;8'h99:d=8'hee;8'h9a:d=8'hb8;8'h9b:d=8'h14;
8'h9c:d=8'hde;8'h9d:d=8'h5e;8'h9e:d=8'h0b;8'h9f:d=8'hdb;

8'ha0:d=8'he0;8'ha1:d=8'h32;8'ha2:d=8'h3a;8'ha3:d=8'h0a;
8'ha4:d=8'h49;8'ha5:d=8'h06;8'ha6:d=8'h24;8'ha7:d=8'h5c;
8'ha8:d=8'hc2;8'ha9:d=8'hd3;8'haa:d=8'hac;8'hab:d=8'h62;
8'hac:d=8'h91;8'had:d=8'h95;8'hae:d=8'he4;8'haf:d=8'h79;

8'hb0:d=8'he7;8'hb1:d=8'hc8;8'hb2:d=8'h37;8'hb3:d=8'h6d;
8'hb4:d=8'h8d;8'hb5:d=8'hd5;8'hb6:d=8'h4e;8'hb7:d=8'ha9;
8'hb8:d=8'h6c;8'hb9:d=8'h56;8'hba:d=8'hf4;8'hbb:d=8'hea;
8'hbc:d=8'h65;8'hbd:d=8'h7a;8'hbe:d=8'hae;8'hbf:d=8'h08;

8'hc0:d=8'hba;8'hc1:d=8'h78;8'hc2:d=8'h25;8'hc3:d=8'h2e;
8'hc4:d=8'h1c;8'hc5:d=8'ha6;8'hc6:d=8'hb4;8'hc7:d=8'hc6;
8'hc8:d=8'he8;8'hc9:d=8'hdd;8'hca:d=8'h74;8'hcb:d=8'h1f;
8'hcc:d=8'h4b;8'hcd:d=8'hbd;8'hce:d=8'h8b;8'hcf:d=8'h8a;

8'hd0:d=8'h70;8'hd1:d=8'h3e;8'hd2:d=8'hb5;8'hd3:d=8'h66;
8'hd4:d=8'h48;8'hd5:d=8'h03;8'hd6:d=8'hf6;8'hd7:d=8'h0e;
8'hd8:d=8'h61;8'hd9:d=8'h35;8'hda:d=8'h57;8'hdb:d=8'hb9;
8'hdc:d=8'h86;8'hdd:d=8'hc1;8'hde:d=8'h1d;8'hdf:d=8'h9e;

8'he0:d=8'he1;8'he1:d=8'hf8;8'he2:d=8'h98;8'he3:d=8'h11;
8'he4:d=8'h69;8'he5:d=8'hd9;8'he6:d=8'h8e;8'he7:d=8'h94;
8'he8:d=8'h9b;8'he9:d=8'h1e;8'hea:d=8'h87;8'heb:d=8'he9;


```
8'hec:d=8'hce;8'hed:d=8'h55;8'hee:d=8'h28;8'hef:d=8'hdf;
```

```
8'hf0:d=8'h8c;8'hf1:d=8'ha1;8'hf2:d=8'h89;8'hf3:d=8'h0d;  
8'hf4:d=8'hbf;8'hf5:d=8'he6;8'hf6:d=8'h42;8'hf7:d=8'h68;  
8'hf8:d=8'h41;8'hf9:d=8'h99;8'hfa:d=8'h2d;8'hfb:d=8'h0f;  
8'hfc:d=8'hb0;8'hfd:d=8'h54;8'hfe:d=8'hbb;8'hff:d=8'h16;
```

```
default:d=8'h00;
```

```
endcase
```

```
end
```

```
endmodule
```

```
module aes_key_expand(  
input [127:0] key,  
output [1407:0] round_keys  
);
```

```
wire [31:0] W [0:43];
```

```
assign W[0] = key[127:96];  
assign W[1] = key[95:64];  
assign W[2] = key[63:32];  
assign W[3] = key[31:0];
```

```
wire [31:0] temp1,temp2,temp3,temp4,temp5,temp6,temp7,temp8,temp9,temp10;
```

```
aes_subword sw1({W[3][23:0],W[3][31:24]},temp1);  
aes_subword sw2({W[7][23:0],W[7][31:24]},temp2);  
aes_subword sw3({W[11][23:0],W[11][31:24]},temp3);  
aes_subword sw4({W[15][23:0],W[15][31:24]},temp4);  
aes_subword sw5({W[19][23:0],W[19][31:24]},temp5);  
aes_subword sw6({W[23][23:0],W[23][31:24]},temp6);  
aes_subword sw7({W[27][23:0],W[27][31:24]},temp7);  
aes_subword sw8({W[31][23:0],W[31][31:24]},temp8);
```

```
aes_subword sw9({W[35][23:0],W[35][31:24]},temp9);  
aes_subword sw10({W[39][23:0],W[39][31:24]},temp10);
```

```
assign W[4] = W[0] ^ temp1 ^ 32'h01000000;  
assign W[5] = W[1] ^ W[4];  
assign W[6] = W[2] ^ W[5];  
assign W[7] = W[3] ^ W[6];
```

```
assign W[8] = W[4] ^ temp2 ^ 32'h02000000;  
assign W[9] = W[5] ^ W[8];  
assign W[10] = W[6] ^ W[9];  
assign W[11] = W[7] ^ W[10];
```

```
assign W[12] = W[8] ^ temp3 ^ 32'h04000000;  
assign W[13] = W[9] ^ W[12];  
assign W[14] = W[10] ^ W[13];  
assign W[15] = W[11] ^ W[14];
```

```
assign W[16] = W[12] ^ temp4 ^ 32'h08000000;  
assign W[17] = W[13] ^ W[16];  
assign W[18] = W[14] ^ W[17];  
assign W[19] = W[15] ^ W[18];
```

```
assign W[20] = W[16] ^ temp5 ^ 32'h10000000;  
assign W[21] = W[17] ^ W[20];  
assign W[22] = W[18] ^ W[21];  
assign W[23] = W[19] ^ W[22];
```

```
assign W[24] = W[20] ^ temp6 ^ 32'h20000000;  
assign W[25] = W[21] ^ W[24];  
assign W[26] = W[22] ^ W[25];  
assign W[27] = W[23] ^ W[26];
```

```
assign W[28] = W[24] ^ temp7 ^ 32'h40000000;  
assign W[29] = W[25] ^ W[28];  
assign W[30] = W[26] ^ W[29];  
assign W[31] = W[27] ^ W[30];
```

```
assign W[32] = W[28] ^ temp8 ^ 32'h80000000;  
assign W[33] = W[29] ^ W[32];  
assign W[34] = W[30] ^ W[33];  
assign W[35] = W[31] ^ W[34];
```

```
assign W[36] = W[32] ^ temp9 ^ 32'h1b000000;  
assign W[37] = W[33] ^ W[36];  
assign W[38] = W[34] ^ W[37];  
assign W[39] = W[35] ^ W[38];
```

```
assign W[40] = W[36] ^ temp10 ^ 32'h36000000;  
assign W[41] = W[37] ^ W[40];  
assign W[42] = W[38] ^ W[41];  
assign W[43] = W[39] ^ W[42];
```

```
assign round_keys = {  
    {W[0],W[1],W[2],W[3]},  
    {W[4],W[5],W[6],W[7]},  
    {W[8],W[9],W[10],W[11]},  
    {W[12],W[13],W[14],W[15]},  
    {W[16],W[17],W[18],W[19]},  
    {W[20],W[21],W[22],W[23]},  
    {W[24],W[25],W[26],W[27]},  
    {W[28],W[29],W[30],W[31]},  
    {W[32],W[33],W[34],W[35]},  
    {W[36],W[37],W[38],W[39]},  
    {W[40],W[41],W[42],W[43]}  
};
```

```
endmodule
```